

UBKOracle Internal Audit Report

Oracle price resolution, invariants, usage modes, and failure analysis

UBKOracle Internal Audit

Scope: UBKOracle.sol and UBKOracleConstants.sol (as provided)

Prepared for internal engineering review ahead of beta tag (0.2.0).

Snapshot date	2025-12-23
Compiler	Solidity 0.8.20
Key dependencies	OpenZeppelin Ownable, Math; Chainlink AggregatorV3Interface; IERC4626; UBKDecimalsBounded
Oracle modes	NORMAL / PAUSED; per-token manual override

Disclaimer. Best-effort internal review only (not a formal security audit).

Contents

- 1. Executive summary
- 2. Invariants
- 3. Usage modes
- 4. Failure modes
- 5. Architecture: waterfall price resolution (diagram)
- Appendix A. Default parameters and bounds
- Appendix B. Suggested hardening patches

1. Executive summary

UBKOracle provides a deterministic pricing layer that normalizes prices to 1e18 precision and supports manual overrides, ERC-4626 vault share pricing, and Chainlink feeds. The design separates **cached reads** from **live updates** to keep hot paths predictable and cheap.

Key strengths

- Decimals are validated once at registration and cached (no runtime decimals() calls).
- Waterfall resolution includes guarded ERC-4626 pricing with recursion limits and sanity bounds.
- Staleness controls exist for both primary (Chainlink) and fallback (last valid) sources.
- Manual mode enables rapid response during feed outages and is bounded (when a recent last-valid exists).

Key risks / recommended follow-ups

- **Admin misconfiguration risk** is the dominant threat model: owner can set feeds, vault underlyings, and manual prices.
- ERC-4626 validation currently does **not** assert `IERC4626(vault).asset() == underlying`; a wrong mapping would misprice vault shares.
- Chainlink best-practice check for *answeredInRound* is not performed; this can accept incomplete rounds on some feeds.
- Default stale windows (24h feed, 48h fallback) are broad; for highly volatile assets, shorter periods may be appropriate.

Summary findings

Finding	Severity	Recommendation
ERC-4626 underlying not asserted	High	In <code>_validateERC4626Vault</code> , require <code>IERC4626(vault).asset() == underlying</code> to prevent misconfiguration-driven mispricing.
Chainlink round completeness not checked	Medium	Validate <code>answeredInRound >= roundId</code> (and optionally <code>startedAt != 0</code>) in <code>_getChainlinkPrice</code> .
Broad default staleness windows	Low	Consider per-asset <code>stalePeriod</code> tuning and a formal keeper cadence (e.g., refresh at least once per <code>stalePeriod/2</code>).

2. Invariants

The oracle's safety relies on invariants being true at all times. Violations typically lead to explicit reverts or mispricing if caused by misconfiguration.

2.1 Registration and support

- **Explicit registration:** tokens become supported only via `_addSupportedToken()`.
- **Uniqueness:** `supportedTokens` contains no duplicates; `isSupported[token]` is canonical.
- **Decimals cache:** `tokenDecimals[token]` is set once and relied upon thereafter.
- **Bounded decimals:** decimals are enforced within `[6,18]` by `UBKDecimalsBounded`.

2.2 Price-domain constraints

- **WAD normalization:** all prices are `1e18` (WAD).
- **Absolute bounds:** resolved/persisted prices must lie in `[1e10, 1e24]`.
- **Freshness for reads:** cached prices must satisfy `age <= stalePeriod[token]`.
- **Freshness for fallback:** fallback requires `age <= fallbackStalePeriod[token]`.

2.3 ERC-4626 vault constraints

- **Vault-underlying mapping:** `erc4626Underlying[vault]` must be correct (critical).
- **Decimals parity:** `vault.decimals == underlying.decimals` (enforced at registration).
- **Recursion bounded:** nested vault resolution depth `<= 5`.
- **convertToAssets sanity:** `assetsPerShare` in `(0, 1e36]`.
- **Rate bounds:** derived rate must be within `[0.2x, 3.0x]` (or custom bounds).

2.4 Manual mode constraints

- **Manual price stored in WAD.**
- **Bounded when recent:** if a recent `lastValid` exists (`age <= stalePeriod`), manual price must be within `+/-10%`.
- **Cache coherence:** manual updates also update `lastValidPrice` to keep reads consistent.

2.5 Conversion constraints (toUSD/fromUSD)

- **toUSD:** normalize amount to WAD via cached decimals, then multiply by WAD price.
- **fromUSD:** divide USD amount by price in WAD-space, then scale to native decimals.
- Both short-circuit to 0 on zero input.

3. Usage modes

UBKOracle's behavior is governed by the global oracle mode (NORMAL/PAUSED) and per-token configuration. Reads are cached; updates are live.

3.1 Global mode

- **NORMAL:** `fetchAndUpdatePrice()` and configuration are available.
- **PAUSED:** update calls and `setManualPrice()` are blocked; cached reads still work while fresh.

3.2 Read vs update path

- **Read path:** `getPrice()/toUSD()/fromUSD()` only use cached state (no external calls).
- **Update path:** `fetchAndUpdatePrice()` resolves live sources and persists to `lastValidPrice`.

3.3 Per-token resolution order

Priority	Source	Activation	Notes
1	Manual override	<code>isManual[token] == true</code>	Set via <code>setManualPrice()</code> . Bounded when a recent last-valid exists.
2	ERC-4626 vault	<code>erc4626Underlying[token] != 0</code>	Price = <code>underlyingPrice * rate</code> . Guarded by recursion, sanity bounds, and rate bounds.
3	Chainlink feed	<code>chainlinkFeeds[token] != 0</code>	Read <code>latestRoundData</code> ; validate positivity and freshness; normalize to <code>1e18</code> .
4	Fallback to last valid	Chainlink invalid	Allowed only if <code>age <= fallbackStalePeriod[token]</code> . Emits <code>OracleFallbackUsed</code> .

3.4 Keeper cadence expectations

- Refresh each listed asset at least once per `stalePeriod/2` (recommended baseline).
- Tune `stalePeriod` per asset class (tighter for volatile assets).
- Alert on `OracleFallbackUsed` and approaching staleness.

4. Failure modes

4.1 Cached-read failures

- **NoFallbackPrice(token)**: token has no cached price (never updated or unsupported).
- **StalePrice(token,...)**: cached price age exceeds stalePeriod[token].

4.2 Live-update failures

- **OraclePaused**: global mode PAUSED.
- **TokenNotSupported**: token not registered in isSupported.
- **NoPriceFeed**: no manual and no ERC-4626, and chainlinkFeeds[token] == 0.
- **RecursiveResolution**: nested vault depth exceeded.
- **InvalidVaultExchangeRate / SuspiciousVaultRate**: convertToAssets or derived rate out of bounds.
- **StaleFallback**: Chainlink invalid and lastValid too old for fallback.
- **InvalidOraclePrice**: resolved price outside absolute bounds.

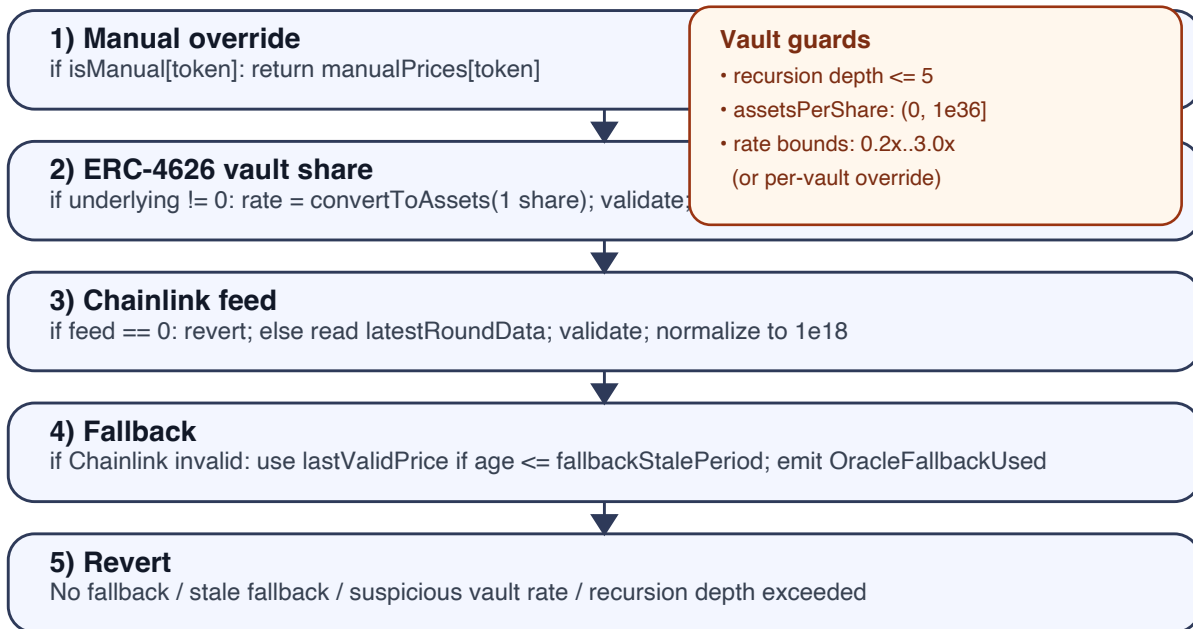
4.3 Controlled degradation

- Chainlink call failures return (0,false) and do not revert immediately.
- If fallback is fresh, Oracle emits OracleFallbackUsed and returns lastValidPrice instead of failing hard.

5. Architecture: waterfall price resolution

The diagram below captures the exact high-level control flow in `_resolvePrice(token)`.

Waterfall price resolution (UBKOracle._resolvePrice)



5.1 Pseudocode

```

function _resolvePrice(token):
  require isSupported[token]
  if isManual[token]:
    return manualPrices[token]
  if erc4626Underlying[token] != 0:
    return underlyingPrice *
    rate(convertToAssets)
  require chainlinkFeeds[token] != 0
  if chainlinkValid:
    return chainlinkPrice
  require fallbackFresh:
    return lastValidPrice
  
```

Appendix A. Default parameters and bounds

Category	Constant	Value	Meaning
Price	ORACLE_MIN_ABSOLUTE_PRICE_WAD	1e10	Minimum allowed normalized price
Price	ORACLE_MAX_ABSOLUTE_PRICE_WAD	1e24	Maximum allowed normalized price
Manual	ORACLE_MANUAL_PRICE_MAX_DELTA_WAD	0.1e18	Max +/- deviation when last valid is fresh
Vault	ORACLE_MIN_VAULT_RATE_WAD	0.2e18	Default minimum vault rate (0.2x)
Vault	ORACLE_MAX_VAULT_RATE_WAD	3e18	Default maximum vault rate (3.0x)
Staleness	ORACLE_DEFAULT_STALE_PERIOD	24 hours	Default Chainlink stale window
Recursion	ORACLE_MAX_RECURSION_DEPTH	5	Nested ERC-4626 depth limit

Appendix B. Suggested hardening patches

```
1) Enforce vault.asset() == underlying during registration. address actual =
IERC4626(vault).asset(); if (actual != underlying) revert ERC4626UnderlyingMismatch(vault,
actual, underlying);

2) Add answeredInRound validation for Chainlink. (uint80 roundId, int256 answer, , uint256
updatedAt, uint80 answeredInRound) = agg.latestRoundData(); if (answeredInRound < roundId)
return (0, false);
```

6. Gas analysis (hot paths)

This section summarizes gas characteristics for the oracle's most frequently used paths when called on-chain. Figures are best-effort estimates based on opcode-level cost structure (including the warm/cold access model from EIP-2929) and typical ERC-4626 / Chainlink feed implementations. Exact costs vary by feed contract, network, and whether storage slots are accessed cold (first touch) or warm (already accessed in the same transaction).

6.1 Cached read paths

Scope: `getPrice()` / `toUSD()` / `fromUSD()` use cached state only (no external calls).

Call path	Storage ops	Estimated gas	Notes
<code>getPrice()</code> → <code>_getPrice(token)</code>	SLOAD: <code>lastValidPrice[token].price</code> ; <code>lastValidPrice[token].timestamp</code> ; <code>stalePeriod[token]</code>	Warm: ~4k–7k Cold: ~10k–16k	Dominated by 3 mapping reads + freshness branch.
<code>toUSD(token, amount)</code>	SLOAD: <code>tokenDecimals[token]</code> + (<code>_getPrice</code> : 3 SLOAD) Early return if <code>amount==0</code>	Warm: ~6k–10k Cold: ~14k–24k	Arithmetic is cheap; storage reads dominate.
<code>fromUSD(token, usdAmount)</code>	SLOAD: <code>tokenDecimals[token]</code> + (<code>_getPrice</code> : 3 SLOAD) Early return if <code>usdAmount==0</code>	Warm: ~6k–10k Cold: ~14k–24k	Same profile as <code>toUSD</code> .

6.2 Live update paths

Scope: `fetchAndUpdatePrice()` resolves a live source and persists `lastValidPrice[token]` (two SSTOREs: price, timestamp) and emits an event.

Update path	Source calls	Storage ops	Estimated gas
Manual mode <code>isManual[token] == true</code>	None	SLOAD: <code>isManual</code> , <code>manualPrices</code> SSTORE: <code>lastValidPrice</code> (2 slots) LOG: <code>LastValidPriceUpdated</code>	Existing: ~30k–45k First init: ~70k–90k
Chainlink feed	STATICCALL: <code>latestRoundData()</code> STATICCALL: <code>decimals()</code>	SLOAD: <code>chainlinkFeeds</code> , <code>stalePeriod</code> (+ fallback reads if invalid) SSTORE: <code>lastValidPrice</code> (2 slots) LOG	Existing: ~60k–95k First init: ~110k–145k
ERC-4626 vault share	STATICCALL: <code>convertToAssets()</code> + underlying resolution (often Chainlink)	SLOAD: <code>erc4626Underlying</code> , <code>vaultRateBounds</code> , <code>tokenDecimals(vault+underlying)</code> SSTORE: <code>lastValidPrice</code> (2 slots) LOG	Existing: ~90k–160k First init: ~140k–210k

6.3 Batch updates

`fetchAndUpdatePrice(address[] tokens)` amortizes the *whenNotPaused* check and calldata decoding, but still performs a full resolve + persist per token. Expect roughly linear scaling: $totalGas \approx base + \Sigma$

perTokenCost, where *perTokenCost* is one of the live update profiles above.

6.4 Notes / cost drivers

- Cold vs warm access is the largest swing on read paths: repeated use of the same token within a single transaction will be cheaper after the first touch.
- SSTORE dominates update paths. Updating an existing price is substantially cheaper than initializing a token for the first time (zero→nonzero).
- Chainlink reads include two external calls in the current implementation (latestRoundData + decimals). Caching feed decimals at registration can remove one call from the hot update path.
- ERC-4626 pricing cost is dominated by convertToAssets plus the underlying resolution (which may itself hit Chainlink). Nesting is bounded by recursion depth.